

Fast Two-Level Address Translation for Virtualized Systems

Jeongseob Ahn, Seongwook Jin, and Jaehyuk Huh

Abstract—Recently, there have been several improvements in architectural supports for two-level address translation for virtualized systems. However, those improvements including HW-based two-dimensional (2D) page walkers have extended the traditional multi-level page tables, without considering the memory management characteristics of virtual machines. This paper exploits the unique behaviors of the hypervisor, and proposes three new nested address translation schemes for virtualized systems. The first scheme called nested segmentation is designed for static memory allocation, and uses HW segmentation to map the VM memory directly to large contiguous memory regions. The second scheme proposes to use a flat nested page table for each VM, reducing memory accesses by the current 2D page walkers. The third scheme uses speculative inverted shadow paging, backed by non-speculative flat nested page tables. The speculative mechanism provides direct translation with a single memory reference for common cases without page table synchronization overheads. We evaluate the proposed schemes with the Xen hypervisor running on a full system simulator. Nested segmentation can reduce the overheads of two-level translation significantly for a certain cloud computing model. The nested segmentation, flat page tables, and speculative shadowing improve a state-of-the-art 2D page walker by 10, 7, and 14 percent respectively.

Index Terms—Virtualization, address translation, nested paging, nested segmentation, flat page table

1 INTRODUCTION

As system virtualization has been widely used for public cloud computing as well as corporate server consolidation, recent processor designs have been extending their architectural supports for virtualization [3], [4], [44]. One of such architectural enhancements for virtualization is to support two-dimensional (2D) page table walks, which can traverse both guest and nested page tables with a hardware page table walker. Translation look-aside buffers (TLBs) store direct translations from guest virtual to system physical page numbers, but for each TLB miss, both page tables must be traversed.

However, the current hardware supports for nested page table walks extend traditional HW-based multi-level page table walkers. Both of the guest and nested page tables are organized as multi-level page tables to reduce memory overheads. With four-level page tables commonly used for 64-bit address spaces, a 2D page table walk takes six times more references than a native page walk [12]. Such multi-level page tables are designed for memory usage patterns for processes in native operating systems, to reduce memory footprints for page tables. However, the traditional multi-level tables are not optimized for the requirement of nested page tables for virtualization.

In this paper, we revisit the design space of hardware-assisted nested paging supports, by considering the unique

characteristics of memory usages and allocation patterns for virtual machines. Based on the characteristics, this paper proposes three schemes to improve nested paging supports, *nested segmentation*, *flat nested paging*, and *speculative inverted shadow page table (SpecISP)*. The nested segmentation mechanism uses the traditional segmentation-based memory management for nested address translation by the hypervisor. With the nested segmentation, the overhead of two-level translation can be reduced significantly. However, the memory for VMs must be allocated to a few chunks of contiguous real memory, limiting the dynamic memory allocation by the hypervisor.

Secondly, flat paging supports page-based fine-grained memory management when dynamic memory management is important. Since the mapping range and usage of VM memory quite differ from those of process memory, the traditional multi-level page organization does not have much space-reduction benefit for nested page tables. Considering the characteristics, we propose that the nested page table can be flattened to a single level page table. By eliminating unnecessary levels, memory references for TLB misses can be reduced significantly. The third mechanism we propose is a speculative mechanism, called *speculative inverted shadow page table*, which provides direct translation from virtual to machine addresses. With the speculative mechanism, each TLB miss looks up the speculative inverted shadow page table, which should take only a single memory reference for common cases. At the same time, the flat page table is accessed to retrieve the correct mapping. A TLB miss is served speculatively, and must be verified by the correct flat page table.

We evaluate the proposed reorganization of nested page tables, nested segmentation, and speculative shadow paging with a real hypervisor running on a full-system simulator. Nested segmentation can almost eliminate the

- The authors are with the Department of Computer Science, Korea Advanced Institute of Science and Technology, 335 Gwahak-ro (373-1 Guseong-dong), Yuseong-gu, Daejeon 305-701, Republic of Korea. E-mail: {jeongseob, swjin, jhuh}@calab.kaist.ac.kr.

Manuscript received 9 Dec. 2013; revised 21 Jan. 2015; accepted 30 Jan. 2015. Date of publication 5 Feb. 2015; date of current version 11 Nov. 2015.

Recommended for acceptance by J. D. Bruguera.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TC.2015.2401022

Authorized licensed use limited to: AJOU UNIVERSITY. Downloaded on September 28, 2023 at 04:34:31 UTC from IEEE Xplore. Restrictions apply.

0018-9340 © 2015 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

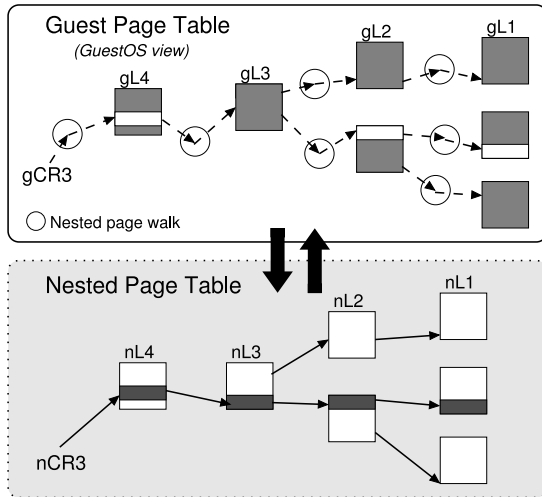


Fig. 1. Nested page table for virtual machines.

overheads of nested address translation, while limiting memory management by the hypervisor. Flat page tables improve a state-of-the-art HW 2D page walker [12] with a page walk cache (PWC) and nested TLB (NTLB) by 7 percent on average for a set of applications. Speculative inverted shadow paging improves the same 2D page walker by 14 percent on average for our benchmark applications [6].

The rest of the paper is organized as follows. Sections 2 and 3 present the background and prior work of hardware-assisted memory virtualization and address translation problems. Sections 4, 5, and 6 present the proposed three schemes. Section 7 presents the experimental results, and Section 8 concludes the paper.

2 BACKGROUND

2.1 Address Translation for VMs

System virtualization provides an isolated memory space for each virtual machine [5], [7], [17], [26]. To provide each VM with such an isolated guest-physical address space, virtualization requires two-level address translations. For each process running in a VM, a guest virtual address (gVA) is translated into a guest physical address (gPA) by a per-process guest page table, which is maintained by the guest OS in the VM. As multiple VMs share the system memory, the guest physical address must be translated into a system physical address (sPA) with a per-VM nested page table. Translation look-aside buffers keep direct translations from guest-virtual addresses to system-physical addresses. For a TLB miss, either a SW or HW page table walker must traverse two page tables to fetch the final translation to the system physical page number. For HW-based page walkers used in many commercial processors including popular x86 architectures, there are two common ways to translate a guest-virtual page number to a system-physical page number for TLB misses without any modification in guest OSes.

In a software-oriented method, for each process in guest virtual machines, the hypervisor maintains a shadow page table, which has a direct mapping between guest virtual to system physical addresses. Shadow paging can be used

with traditional page table walkers which can traverse only a guest page table for a TLB miss, and it does not require any extra architectural support for virtualization. Each core has a register pointing the top-level page table (CR3 in the x86 architecture). Whenever a guest OS changes the page table register to point a new guest page table, the hypervisor must intercept the change, and update the register to the corresponding shadow page table. Per-process page tables must be duplicated in the guest OS and hypervisor, and furthermore, any change of a page table by the guest OS requires a costly hypervisor intervention to update the corresponding shadow page table. For applications with frequent memory mapping changes, such synchronization overheads between guest and shadow page tables cause significant performance degradation [48].

2.2 Hardware-Assisted Virtualization

Recent supports for two-dimensional walks eliminate the need for shadow paging. Each core has both the page table pointer to the current guest page table (gCR3 in x86), and the page table pointer to the nested page table (nCR3 in x86). Unlike a shadow page table which translates from virtual address to system physical address space directly, the nested page table translates from virtual physical to system physical address space. For a TLB miss, the HW walker traverses both of the guest and nested tables to get the final translation between guest-virtual and system-physical page numbers. Such page table walkers are designed to support the current multi-level page tables, whose formats are specified and fixed in the instruction set architecture. The hardware walker assumes the same organization of page tables for guest and nested page tables, although page granularity can differ modestly. Due to the restricted page table organization for multi-level tables, a TLB miss can cause many references to both page tables.

Fig. 1 describes guest and nested page tables. A guest page table for a guest process contains only virtual physical addresses, and all the addresses in each level must be translated to machine addresses through the per-VM nested page table. Fig. 2 presents the address translation process for virtualized systems in the 64-bit long mode of the x86 architecture. In the figure, the rectangles are guest page table walks, and the circles are nested page table walks. The $gL4$ and $nL4$ indicate the top layer of indexing the guest and nested page tables, respectively. The $gL1$ and $nL1$ mean the bottom layer of guest and nested page tables. Native systems require only 4 references to the page table for each TLB miss, but virtualized systems requires 24 references to the two tables by walking the tables in a two-dimensional way. More generally, if guest page tables and nested page tables are m and n levels, the 2D page walks take $mn+m+n$ references [12]. Due to the long latencies for handling TLB misses, the performance of applications with high TLB miss rates may be even lower with the nested page table walker than that with shadow paging [45], [48]. On the other hand, the hardware-assisted 2D page walker performs better than shadow paging for applications with frequent updates on guest page tables. Furthermore, the hardware-assisted mechanism does not require extra memory for duplicating page tables for guest processes.

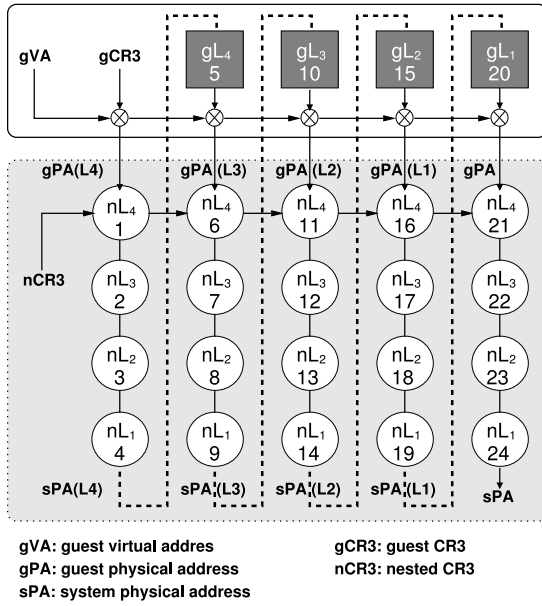


Fig. 2. Two-dimensional page walks for virtualization in x86 architectures.

To reduce the overhead of many memory references by 2D page walkers, some architectures use a *page walk cache*, which is an extra hardware table to hold intermediate translations [3], [12]. For a TLB miss, instead of accessing the memory, a small hardware PWC is first checked to find the necessary intermediate translations. As upper-level intermediate translations, which cover a large memory space in multi-level paging, exhibit high temporal localities, a small number of entries in PWC may capture the working set of the upper-level translations. The rectangles and circles shown in Fig. 2 can be cached in PWC. In 1D PWC, intermediate translations only for guest page tables (some of the rectangles in the figure) are cached in PWC, and in 2D PWC, intermediate translations both for guest and nested page tables (some of the rectangles and circles) can be cached in PWC. However, with 2D PWC, the limited PWC capacity must be shared by the intermediate translations for 1D translation from guest virtual to guest physical pages, and the 2D intermediate translations for nested page table walks. Bhargava et al. describe the details of the caching policy [12].

Another hardware support to reduce the overhead of walking 2D page tables is *nested TLBs* [3]. The nested TLBs hold the most recently used mappings from guest-physical to system-physical pages, as shown in Fig. 2. These two techniques, PWC and NTLB, are used to reduce the overhead of multi-level nested page tables. The overhead of multi-level page walks stems from the assumption that nested page tables have the same requirements as the traditional page tables used in native OS systems. Fig. 3 presents the performance improvements by PWC and NTLB. Perfect-TLB means that there is no penalty for the TLB miss. The detailed methodology for this evaluation is described in Section 7.1. As shown in the figure, 2D PWC provides the most significant performance improvement, and NTLB further improves the performance slightly. In the rest of the paper, we will use both 2D PWC and NTLB in our baseline system to compare the proposed schemes.

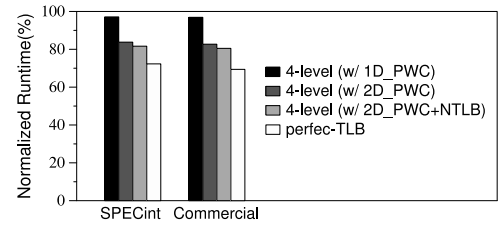


Fig. 3. Normalized runtimes with PWC and NTLB. 1D and 2D denote the caching policy of page walk caches (1D PWC and 2D PWC).

3 RELATED WORK

This paper extends our prior study with nested segmentation support and the analysis and evaluation of large pages and sensitivity studies for completeness [6]. Reducing the overheads of address translation has been a critical aspect of optimizing the memory hierarchy in native systems as well as virtualized systems [11], [21], [33]. For native systems, several studies have investigated schemes to reduce TLB misses or reduce the overheads of handling TLB misses.

One approach to reduce TLB misses is to support multiple page sizes [34], [37], [39], [40], [41], [42], [43]. Talluri et al. investigated several existing page table schemes in a 64-bit system and proposed the clustered page table which exploits contiguous memory allocation [42]. Ganapathy and Schimmel investigated the operating system support for using multiple page sizes [24]. A key component for supporting multiple page sizes is how to promote page sizes statically or dynamically [23], [37]. Romer et al. presented a technique dynamically promoting normal pages to a large page [37]. Jacob and Mudge evaluated various memory management units including software and hardware-based mechanisms [29]. Navarro et al. discussed the advantage and disadvantage of large pages and designed memory management systems for large pages [34].

Barr et al. compared several designs of caching intermediate page table entries in multi-level page tables, evaluating various page walk caching policies [8]. Another work by the same authors, SpecTLB, proposed to use speculation for address translation to reduce the page walk overhead [9]. It exploits the spatial locality available in memory allocation policies in commercial operating systems. Speculative shadow paging proposed in this paper has been inspired by their speculative mechanism for handling TLB misses, although the purpose of using speculation greatly differs from that for SpecTLB. To cluster multiple TLB entries into a single entry, the coalescing technique was introduced by operating system supports [35], [36]. TLB prefetching has also been studied to mitigate TLB miss costs [30].

Virtualization has made efficient address translation critical for the overall system performance, as it requires both guest and nested page table translations. Bhargava et al. discussed page walk overheads in virtualized systems [12]. They showed that a small number of entries for nested page tables are frequently reused due to spatial and temporal localities. To take advantage of this characteristic, page walk caches used in non-virtualized systems are extended to cover nested page walks. In addition, they also evaluated nested TLBs to further reduce memory

TABLE 1
Memory Configurations in Commercial Clouds

	Available memory configurations (GB)
Amazon EC2	1.7, 3.75, 7.5, 15, 30
Microsoft Azure	0.75, 1.7, 3.5, 7, 14
Google	3.75, 7.5, 15, 30, 60
Rackspace	1, 2, 4, 8, 16

references for nested translations. Wang et al. proposed a selective address translation mechanism between software-based shadow paging and hardware 2D page walk [48]. Unlike prior studies, we focus on redesigning the page tables by exploiting memory usage patterns in virtualization. Chang et al investigated how to improve memory virtualization when a software-managed TLB is used instead of hardware page walkers [19]. A recent study published in parallel with the revision of this paper proposed a nested segmentation to provide segment-based translation for both guest and host translation [25].

There are several studies to reduce TLB misses in chip multiprocessors. The SPARC architecture proposed translation storage buffers (TSB) to directly supply the address translation as software MMU [2]. Recently, Bhattacharjee and Martonosi investigated the TLB behaviors of parallel workloads [15]. Their study shows that many TLB misses for each core are redundant and predictable, if the cores run a shared-memory parallel application. To exploit these characteristics, they proposed prefetch mechanisms, which are based on the predictable behaviors [16]. In addition, recent studies proposed the shared last-level TLBs [14] or shared MMU caches [13] which are analogous to shared last-level caches in multi-cores. Shared TLBs maximize the caching efficiency of TLBs by sharing limited physical resources among cores. Synergistic TLBs apply similar techniques used by cooperative private caches to design distributed shared TLBs efficiently [38]. Lustig et al. proposed cooperative prefetching of TLB entries exploiting the cooperative nature of multi-threaded applications [31].

Traditionally, the segmentation is commercially available in PowerPC and IA-32 [22], [28]. Although the original segmentation was invented several decades ago, there is a recent effort to leverage the memory allocation contiguity in certain applications such as large data processing by using segmentation [10].

4 EXPLOITING ADDRESS SPACE CONTIGUITY

Our first scheme, nested segmentation, exploits the static nature of memory allocation. This scheme can almost eliminate the performance overheads of nested address translation, although the dynamic memory management for VMs is not allowed.

4.1 A Case for Static Contiguous Memory for VMs

One of the most distinct characteristics of memory management for virtual machines by the hypervisor is its static nature of allocation. In many important virtualized systems, the hypervisor allocates a fixed memory size to each VM,

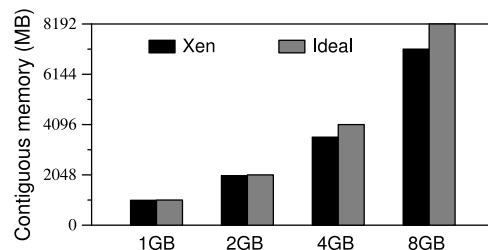


Fig. 4. Contiguously allocated region of a VM.

and does not dynamically change the memory mapping during the life time of the VM. Such a static memory management is commonly used in public cloud services for infrastructure-as-a-service (IaaS). Table 1 presents the memory configuration for popular commercial IaaS platforms. In such cloud computing services, each VM can use only one of the few pre-configured memory sizes, and the memory allocation does not change while the VM is running. The service provider must guarantee the fixed memory capacity to meet the service level agreement (SLA) with the customers, not being able to change the initial allocation at all. Furthermore, dynamic memory management can affect the performance of VMs and cause performance interference among VMs sharing the same physical system.

In the IaaS platforms, since each provider supports only a few possible memory capacity options with commonly a multiple of a large chunk of 512 MB or 1 GB, the real system memory can be easily managed at the granularity of the large chunk. The hypervisor can divide its real memory space into several chunks of the large contiguous memory regions, and assign one or multiple chunks of the memory to each VM with requested memory capacity. Such a simplified memory allocation for VMs suffices for the cloud computing services, and can provide large contiguous memory space for VMs.

Furthermore, even without such a forced memory assignment, the current unmodified hypervisors also attempt to assign contiguous memory to each VM as much as possible. For example, the Xen hypervisor uses the buddy policy to allocate the memory for guest VMs. The hypervisor initially allocates several large chunks of contiguous memory to each virtual machine. Unless dynamic memory management techniques such as ballooning are used, the initial allocation does not change, allowing each VM to maintain the contiguous real memory.

We conducted experiments with the Xen hypervisor to see how many memory regions could be allocated contiguously. To show the memory allocation by the unmodified hypervisor, Fig. 4 presents the memory contiguity of a virtual machine with 1, 2, 4 or 8 GB memory after booting and warming up. The experiment used an Intel Xeon E5330 system equipped with 48 GB physical memory. The large portion of memory was contiguously allocated even if the size of memory is increased. There are many opportunities for allocating contiguous pages of memory because the requested memory size of VMs is relatively larger than traditional processes. Unlike the operating system with many processes requiring dynamic memory management, the hypervisor needs to handle relatively infrequent VM creations with fixed memory requirements in cloud computing,

and thus does not suffer significantly from the fragmentation problem which disturbs the memory contiguity.

As shown in the figure, without dynamic memory management techniques such as ballooning [47], the virtualized systems can maintain a large contiguous memory regions for each VM. With such a static memory allocation with only a few possible memory capacity choices for VMs, two-level address translation can be greatly simplified with nested segmentation, as proposed in the next section.

4.2 Nested Segmentation Architecture

If the hypervisor can allocate large chunks of memory for VMs, the translation through a fine-grained page table is not necessary. To exploit such simple, mostly static, memory allocation policies used for VMs, nested segmentation can replace the conventional translation mechanism through nested page tables. Fig. 6 depicts the organization of nested segmentation. Each core supports several registers, with each register representing a nested segment. Guest operating systems still use a guest page table for each process, and a virtual address is translated to a guest physical address with the table. A guest physical address for each level of translation with guest page tables, is translated into a machine address through one of the segment registers. Each segment register holds the starting address in guest physical address space, the limit of the segment, and the starting address of the segment in the system physical address space. For a guest-physical page number, the segment registers are searched simultaneously to find the corresponding segment. Each entry corresponds to a chunk of memory in a contiguous machine address space. Once a matching segment is found, the machine address for a guest physical address is constructed with the starting machine address of a segment and the offset in the guest physical address.

This nested segmentation eliminates the translation through nested page tables entirely. The latencies of accessing nested segment registers are no longer than a couple of processor cycles, which are negligible compared to the latencies for memory references. Finding a segment for a guest physical page number requires an arithmetic comparison operation for each entry. Due to the cost of such logic in addition to the slight increase of context switching costs to write segment registers, the number of segments should be limited to a few segments.

Providing the hypervisor with the option of using the segment-based nested translation improves the overall performance for certain cloud applications of virtualization with negligible negative impacts. Although using nested segmentation is not always possible for any virtualized system, the IaaS cloud computing model which can allocate a large chunk of contiguous memory statically for each VM, can benefit from this optional hardware support. Adding this mechanism requires only a small increase of design complexity and area for implementation.

4.3 Nested Segment with Partial Paging

The limitation of nested segmentation is that the memory of a VM must consist of only a few chunks of contiguous memory regions. However, hypervisors cannot always find contiguous chunks of memory due to fragmentation.

Furthermore, if the memory usage of a VM fluctuates severely, only a small portion of a segment may be used. To mitigate the restriction of the strict nested segmentation, we propose flexible dual addressing modes. Traditional nested paging can be combined with nested segmentation. The similar scheme was introduced to a native system [10]. If a guest physical address resides beyond segmented regions, traditional page tables can be looked up for complete address translation.

To support such flexible nested segmentation, each segment can support either direct segmented translation to system physical address space, or in-direct translation through a partial page table. A partial page table covers the range of memory specified only in the segment. In Fig. 6, the last segment uses the base field as the starting address of a partial page table for the segment. In the figure, the system uses a flat page table for nested paging, which will be discussed in the next section. With the combination of segmentation and partial page tables, the hypervisor opportunistically tries to find a contiguous region of memory for a segment, and if not available, it can always use a partial page table.

With this combined mechanism, both pure page-level nested paging and pure nested segmentation can be supported. By allowing both direct and indirect translation in segments, nested segmentation allows certain parts of guest physical memory space to be managed at fine granularity. The hypervisor can reserve some portion of memory for large granularity allocation for segment-based mapping, and maintain the rest of memory as fine granularity allocation by partial page tables.

4.4 Effects of Memory Ballooning

If a VM can be assigned with a single contiguous real memory, nested segmentation can eliminate the performance overheads of nested address translation, since it adds just a few CPU cycles for each TLB miss. However, in this section, we consider less ideal cases, and quantify the effect of dynamic memory management on the nested segmentation. Assuming the dual-mode address translation with both nested segmentation and partial paging, we measure how many segments are necessary even with limited page-level dynamic memory management.

Ballooning is a common dynamic memory management technique in hypervisors, which allow guest OSes to determine victim pages. The memory ballooning technique allows the hypervisor to support overcommitment of memory capacity, running more VMs even if the aggregate memory size of the VMs exceeds the physical memory capacity. With ballooning, each guest VM relinquishes unused memory pages to the hypervisor voluntarily. Each VM can be configured to have minimum reserved memory, which is not affected by the ballooning technique [46].

This section shows that even if ballooning is used, a relatively large amount of memory is still statically allocated during the VM creation. We measure how ballooning affects the contiguity of memory for each VM by generating traces for page allocation and deallocation via Xentrace¹. To

1. Xentrace is tracing utility provided by Xen hypervisor which is enable to capture what is going on inside the Xen hypervisor.

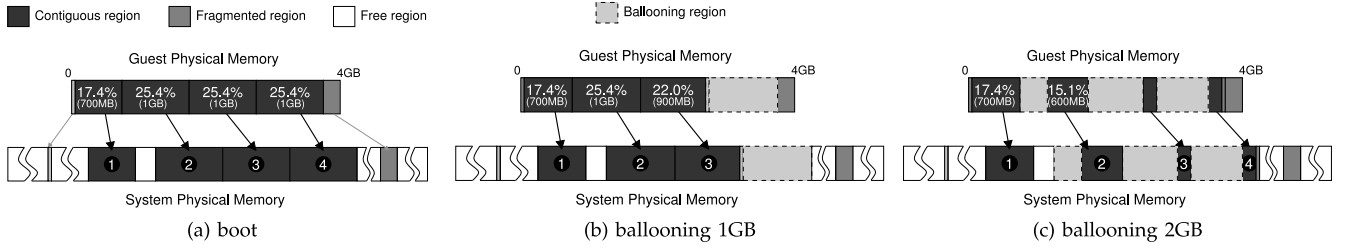


Fig. 5. Memory allocation layout for a virtual machine.

investigate the behavior of balloon, we do not run any applications after booting except for balloon driver. The total memory of the VM is 4GB, and Figs. 5b and 5c show the memory allocation layout after the balloon driver works on 1 and 2 GB memory region of the total 4GB memory, respectively. As you noticed, the fragmentation is inevitable for the several large chunks of contiguous memory. However, it still retains some continuously large chunks for the remaining memory, which is reserved areas for the VM, these are not reclaimed to the hypervisor until the VM is stopped.

As shown in the figure, although some of the initially contiguously allocated regions are fragmented, the continuity of the other regions is still preserved. These memory regions can be handled by the nested segmentation without the address translation incurring the memory reference. Table 2 presents the ratio of contiguous memory in a VM with 4 GB memory, when ballooning is used for 1, 2, and 3 GB of the total memory. As more memory is configured to use ballooning, the contiguous memory ratio decreases. However, with 2GB for ballooning, half of the memory is still preserved as contiguous memory region.

5 FLATTENING NESTED PAGE TABLES

To support a fine-grained dynamic memory management for VMs, page-level mapping is still necessary for certain virtualized systems, which cannot be supported by nested segmentation effectively. In this section, we discuss the differences of memory management for virtual machines and

for processes in native systems. Based on the observations, we propose to reorganize nested page tables to flat tables by exploiting the unique characteristics of memory management for virtual machines.

5.1 Memory Overheads of Nested Page Tables

Current nested page organizations assume that virtual machines are analogous to processes in native systems. However, there are several differences between two entities, which can affect the requirements for nested page tables significantly. In native systems, there are many processes, which may not always be running, but consume the memory to maintain the state. Since a page table must be created for each process even if the process uses a small amount of memory, the total memory used for all page tables must be reduced as much as possible. Furthermore, page tables for processes map the virtual address space to the physical address space. The virtual address space of a process is specified and fixed in the ISA, and it is much larger than the total system memory. For example, in the 64-bit x86 architecture, the long mode assumes a 48-bit virtual address space, which can be as large as 256 TB of memory. Due to the large virtual address space, page tables are organized into multi-level tables. In the 64-bit x86 architecture, a page table is organized as a four-level table, which can save the memory for each page table significantly.

However, virtual machines exhibit quite different behaviors than processes. Firstly, the number of virtual machines in a physical system is limited. For VMs running compute-intensive workloads, the number of virtual CPUs for all guest VMs often does not exceed the number of physical cores. Even for I/O intensive servers with relatively low CPU utilization, the number of VMs in a system is orders of magnitude smaller than the number of processes in native systems.

Secondly, nested page tables, which map the guest physical to system physical memory space, have much smaller mapping ranges than those of the page tables for processes. A per-process page table must cover the entire virtual address space, but a nested page table needs to cover only the guest physical memory space. When a VM is created, the guest physical memory is specified. However, the total

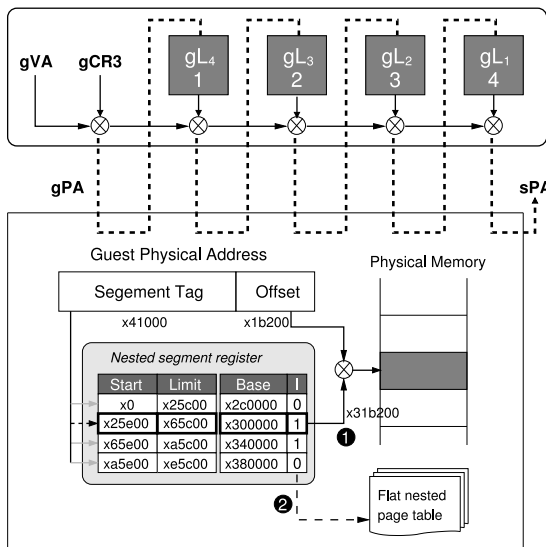


Fig. 6. Nested segmentation with four segments.

TABLE 2
Contiguous Region Ratio with Ballooning

	w/o balloon	1GB	2GB	3GB
contiguous	94.43 percent	67.79 percent	48.92 percent	15.32 percent
# segments	4	3	4	4

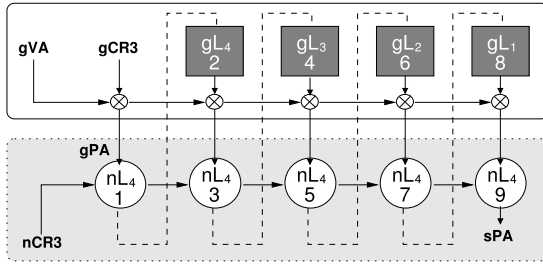


Fig. 7. 2D page walks with a flat nested page table.

guest physical memory from all VMs often does not exceed the available system memory. Even if the VM memory is over-committed, or the total VM memory exceeds the system memory, the overcommitment ratio is commonly two or three times of the system memory size. Furthermore, some hypervisor implementations do not even support the overcommitment of the total VM memory [1].

Thirdly, unlike processes in a native system, most of which use a tiny fraction of virtual memory space, virtual machines use much of the guest physical memory. When a VM is created, an administrator must consider the memory requirement for the VM, and attempt to assign an appropriate memory size. If the majority of the guest physical memory space is used, multi-leveling the nested page table for the VM does not reduce the memory overhead for the nested page table.

Due to the three major differences of virtual machines from processes, multi-level nested page tables do not have significant memory savings for virtual machines. The number of nested page tables is relatively small, as the number of VMs is limited. Each nested page table needs to cover a much smaller range of the guest physical address space than that of the virtual address space. Furthermore, VMs use much of the guest physical space, and thus require the nested page table entry anyway for the space. Therefore, for many of the common use cases of virtual machines, multi-level nested page tables are not necessary, and they only increase the number of references unnecessarily in two-dimensional page walks.

5.2 Flat Nested Page Tables

In this section, exploiting the unique property of VM memory usage, we propose a technique to reduce the overhead of nested page table walks. The technique, called *flat nested page table*, reduces the number of memory references for nested walks by eliminating intermediate page walks. Fig. 7 depicts how the technique simplifies page walks for virtualized systems. Unlike nested segmentation, this scheme supports fine-grained dynamic memory management, since the unit of memory allocation is a page.

As discussed in Section 5.1, the benefit of multi-level page tables is marginal for nested page tables, and thus the hardware page walker can simply use a flat page table walk for nested page tables. Flat page table walks are much simpler than multi-level page table walks, and the support for them can be added to the current multi-level page table walker without any significant increase of complexity. The nested page table pointer (nCR3) has the starting address of the flat table. The address of the missed page table entry is calculated directly by adding the offset to the nCR3 value.

Authorized licensed use limited to: AJOU UNIVERSITY. Downloaded on September 28, 2023 at 04:34:31 UTC from IEEE Xplore. Restrictions apply.

The technique does not necessarily replace the current multi-level nested page table walkers. Instead, the technique can co-exist with the current walkers. For certain virtualized systems, which require multi-level nested page tables, the conventional walker can be used. If such a fine-grained memory management is not necessary, the proposed technique can be used to improve the system performance. The flat nested page tables can be added on top of the currently available HW 2D walkers with a negligible increase of complexity.

Fig. 7 depicts page walk procedures with flat nested page tables. When the level of a guest page table is m , the number of memory references is reduced to $2m+1$ with a flat nested page table. In the four-level page table walks in the 64-bit x64 architecture, a TLB miss requires only 9 references with the flat nested page table, reducing 15 references from the traditional 24 references with a four-level nested page table.

As discussed in Section 5.1, even if a flat nested page table is used for each VM, it does not increase the memory overhead for nested page tables significantly. Only a limited number of VMs run in a system. The flat page table size for a nested page table for a VM is much less than that for a guest page table for a process, since a nested page table needs to cover only the guest physical memory space. For example, for a VM with a 4 GB guest-physical memory, if 4 KB page size is used, the size of a flat nested page table is 8 MB with 8B for each entry. However, for a process with a 48-bit virtual address space, the flat page table size is 512 GB, and thus supporting such a flat page table consumes a lot of physical memory [6].

Page walk cache policy. Nested page table entries are still cached in NTLB, but without any intermediate entries with flat tables, the entire capacity of PWC can be used for guest page table walks (1D PWC).

Supporting large pages: Supporting large pages is also possible with flat page tables. A flat page table has the same number of entries for the smallest supported page size, regardless of actual page granularity. For a large page, all the entries corresponding to the large page are marked as a large page entry. Only the first entry among the multiple entries for the large page, has an actual physical frame number. For a TLB miss, the page number at the smallest page granularity is used to access the flat page table. If the entry is not the first entry, an additional page table access occurs to fetch the physical frame number from the first entry of the large page. Such indirect accesses are necessary, as the page size is unknown when a TLB miss occurs.

Supporting flat page tables does not require any significant changes from current multi-level nested page table walks. Therefore, a system may support flat nested tables as well as multi-level nested tables, and the hypervisor may select different page table types for VMs considering their memory sizes and usage patterns.

6 SPECULATIVE INVERTED SHADOW PAGING

Although flat tables reduces memory references for nested page table walks, they do not reduce the overhead of guest page tables. An alternative page table organization is an inverted page table, which can potentially fetch a page table entry by a single memory reference. In this section, we

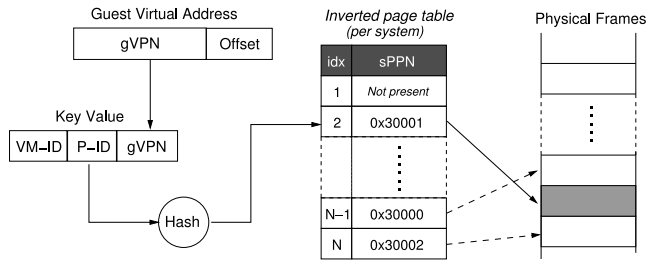


Fig. 8. Hashed inverted shadow page table.

propose an inverted shadow page table, which can retrieve a direct mapping from a guest-virtual page to a system-physical page.

6.1 Trade-Offs of Shadow Paging

Shadow paging provides direct translations from guest virtual to system physical pages, which can potentially eliminate walks for nested page tables. However, maintaining shadow page tables incurs significant performance and memory overheads, often exceeding its benefits. In this section, we discuss three limitations of shadow paging about synchronization costs, extra memory overheads, and multi-level page walks.

The primary performance overhead of shadow paging is the cost of hypervisor interventions. Exiting from a guest VM context to the hypervisor context (*vm-exit*) incurs a significant performance overhead in virtualized systems. If a page fault occurs, the hypervisor must intervene and may have to traverse both the guest and shadow page tables, since it does not know whether the page fault must be handled by the guest OS or hypervisor. Furthermore during a process context switch within a guest VM, the hypervisor should replace the guest page table pointer with the corresponding shadow page table pointer, so that the HW page table walker uses the shadow page table, instead of the guest page table. In addition to the performance overhead, shadow paging incurs the memory overhead for storing extra shadow page tables for each guest process. In addition, the shadow paging is also based on multi-level page tables. The details of performance and other overheads of shadow paging is discussed by Ahn et al. [6].

6.2 Hashed Inverted Shadow Page Tables

One of the traditional alternatives to multi-level page tables is a hashed inverted page table [8], [29]. A hashed inverted page table organizes a page table as a hash table, and searches a page table entry by a hash of the virtual page number and process identifier. One inverted page table is necessary for the entire system, and its size is often proportional to the system memory size. PA-RISC supports such a hashed inverted page table [27]. With the hashed table, the best case requires only one or two memory references if there is no hash collision. However, a collision case may require to search the inverted page table with a chain of many references. A slightly different approach from the inverted page table is to use a directly accessible in-memory cache of the complete page table. translation storage buffers used by the SPARC architecture is an in-memory direct-mapped cache of the complete address translation [2].

A similar directly indexed translation caching can be used for virtualization. Traditional shadow paging uses a duplicate shadow page table for each guest process. Instead of such a per-process shadow page table, we propose a system-wide hashed inverted shadow page, indexed by a hash of the VM identifier, process ID, and guest virtual page number as shown Fig. 8. If the corresponding hashed shadow page entry contains the mapping, a TLB miss can be handled by a single memory reference. If the entry does not contain the requested mapping, a page walk for the complete guest and nested page tables is initiated. As the hashed shadow page is a translation cache for nested translation, it does not replace nested page tables, which are necessary to keep a complete memory mapping for each virtual machine. Note that unlike traditional inverted page tables, the proposed inverted shadow page table does not include a tag in each entry to verify VM identifier, process identifier, and virtual page number. It can eliminate the tag information, since the translations in the inverted page table will be used only speculatively, as discussed in the next section.

The inverted shadow page table does not require the modification of the current page table organization specified in the ISA. However, the hardware page walker must be modified to additionally support a lookup mechanism for the inverted shadow page table. Since the inverted shadow page table is an in-memory cache of the complete translation, the hardware page walker must update the inverted page table if the translation is not in the inverted page table for a TLB miss. The hypervisor only needs to reserve the system memory for the inverted shadow page table. As a system-wide cache of translation, the inverted shadow page may reduce the memory overhead for duplicating all guest page tables in the traditional shadow paging. Even if the number of VMs or vCPUs increases, the size of the system-wide inverted shadow page table does not need to increase. With such a hardware-based management of the inverted shadow paging, it requires only one memory reference to fetch a direction translation from a guest virtual page to a system physical page, if the request hits on the inverted table.

However, such an inverted shadow page table has the same critical problem as the traditional shadow page table. If a guest OS modifies a guest page table, the hypervisor must intervene to update or invalidate the cached entry in the inverted shadow page table. In this paper, we propose a speculative mechanism to eliminate such synchronization by the hypervisor completely.

6.3 Speculatively Handling TLB misses

Using speculation to reduce the cost of TLB misses is not new. SpecTLB first proposed to use speculation to predict address translation [9]. For a TLB miss, the processor continues to execute with a predicted translation, while a traditional page walk will verify the prediction. The speculative mechanism uses the same recovery mechanism commonly found in the current processors supporting speculative executions with re-order buffers or checkpointing [20].

In this paper, we use speculation to eliminate the synchronization between guest page tables and the inverted

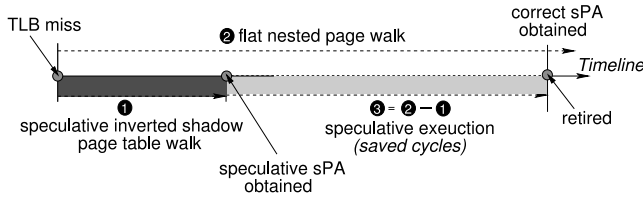


Fig. 9. Speculative page table walk with SpecISP.

shadow page table. *Speculative inverted shadow paging* eliminates the needs for hypervisor interventions, even if a guest page changes. Instead, the inverted shadow page table is allowed to have the obsolete mapping information. For a TLB miss, the processor can speculatively execute with a mapping found in the inverted shadow page table. Simultaneously, the nested page walks will fetch the correct translation to verify whether the speculation is correct. Although the basic speculative execution capability is already supported in the current out-of-order processors, the speculative execution support for TLB misses must be added as another speculative execution event in addition to the correction mechanism.

Fig. 9 depicts the simultaneous walks for the speculative inverted shadow page table (1), and for the non-speculative flat nested page table (2). The speculative walk will finish much earlier than the non-speculative walk for the majority of TLB misses, as it requires only a single memory reference. However, it is also possible that the inverted shadow page table lookup may take longer than the non-speculative flat table lookup. If the lookup of the inverted shadow page table is a cache miss, and all the memory references for the guest and nested page table lookups are cache hits, the speculative lookup may take longer than the non-speculative lookup. In Section 7.4, we will show differences in latencies between speculative and non-speculative page walks with our simulated system.

If a speculative execution is correct, the inverted table organization allows SpecISP to retrieve a translation entry with a single memory reference, in contrast to multiple memory references in 2D page table walks with HW-based nested paging, or 1D page walks with shadow paging. As misspeculation rates are relatively low for many applications, SpecISP excels both HW 2D walking and shadow paging for common cases.

Even if a misspeculation occurs, the latency for a TLB miss is bounded by the latency of the correct nested page walk and the recovery cost for speculative execution. SpecISP excels shadow paging when changes in guest page mappings occur frequently. Misspeculation costs with SpecISP are much smaller than those of hypervisor interventions in shadow paging. A page mapping change will cause SpecISP to misspeculate due to the obsolete information in the inverted table. However, unlike shadow paging which requires hypervisor interventions with long latencies, such a misspeculation can be resolved quickly in SpecISP, since a simultaneous correct nested page walk will finish quickly with the HW-based nested page walker.

SpecISP does not require significant extra supports from conventional speculative execution cores. The conventional speculative execution capability of out-of-order cores for branch and memory speculation is enough to support

SpecISP. The added logic is to compare the speculative system physical address and permission bits with the correct translation when the non-speculative nested page table walk completes for a TLB miss. For a simple in-order processor, it is necessary to add a register checkpointing mechanism and a store queue to support speculation.

7 EVALUATION

7.1 Methodology

Regrettably, the simulation infrastructure for evaluating the virtualized systems was not widely discussed. Many prior studies in architectural extensions with virtualization tended to ignore the effects of a hypervisor layer and did not properly model the hypervisor in their simulation study. In our study, to improve the accuracy of the evaluation, we use the Simics full-system simulator [32], running the real Xen hypervisor (version 4.0.1). By using the real hypervisor, we can include the effects of a hypervisor on TLB and cache behaviors. We also built a custom memory hierarchy model. It has been added to the Simics simulator to model multi-level caches and TLBs on a x86 system. The simulated system has a single core to reduce simulation times, and the core has 32KB L1 instruction and data caches, and a 512KB L2 cache. The system uses separate two-level TLBs for instruction and data. The separate L2 TLB for each instruction and data has 512 entries. All page table entries, including intermediate translations, can be cached in the L2 cache.

We use a simple in-order processor model for the x86 ISA, due to the limitation of our simulation infrastructure. To the in-order model, we have added a checkpointing mechanism for speculative execution. However, if out-of-order execution cores supporting speculative execution, are available, it is not necessary to add such an extra checkpointing mechanism. For a TLB miss, a register checkpoint is created, and during a speculative execution period, up-to 24 stores can be buffered in the store buffer. If no more stores can be buffered in the store buffer, the core is stalled. The speculative execution capability limited by the store buffer size, is similar to the latest out-of-order execution cores with more than 100 outstanding instructions and 24 stores [18].

The baseline system (4-level 2D PWC+NTLB) uses four-level two-dimensional page walks for both guest and nested page tables as shown in Fig. 2. To compare the proposed schemes to the latest advancements for reducing TLB miss latencies, the baseline system has a page walk cache, which can cache 2D intermediate page table entries, and a nested TLB (NTLB). We use the same PWC and NTLB management policies as discussed in Bhargava et al [12]. The details of the system configurations are shown in Table 3.

On top of the Xen hypervisor, a guest virtual machine, which uses a Ubuntu distribution based on Linux kernel 2.6.18, and the domain 0 virtual machine are running. The domain0 virtual machine is a special virtual machine, which handles I/O devices. On the guest VM, we run our benchmark, seven applications selected from the SPECint 2006 benchmark (SPECint), and seven commercial applications (commercial). Table 4 describes the details of benchmark applications.

TABLE 3
Simulated System Configurations

Parameter	Value
Processors	In-order x86 processor
L1 I/D Cache	1-cycle, 32KB, 4-way, 64B block
L2 Cache	12-cycle, 512KB, 8-way, 64B block Average L2 miss latency: 100 cycles
Instruction TLB	1-cycle, 32-entry, fully assoc. L1 2-cycle, 512-entry, 4-way, L2
Data TLB	1-cycle, 64-entry, fully assoc. L1 2-cycle, 512-entry, 4-way, L2
Page Walk Cache	24-entry, fully assoc. 2-cycle PWC access Flushed on each TLB flush
Nested TLB	16-entry, fully assoc. 2-cycle NTLB access Never flushed during guest execution

The execution times shown in the section are all normalized to the execution time with a state-of-the-art hardware 2D page walker with both PWC and NTLB (baseline). The flat nested walker (flat) uses 1D PWC and NTLB. We also show an ideal configuration with the perfect TLBs (perfect-TLB), which does not have any TLB misses. Speculative inverted shadow paging can use either flat page tables (SpecISP w/ flat) or 4-level nested page tables (SpecISP w/4-level), as non-speculative backing page tables.

7.2 Nested Segmentation

Fig. 10 shows normalized execution times with flat page tables, and two configurations with nested segmentation. The nested seg configuration assumes that the allocated memory regions from the hypervisor are perfectly contiguous, and with this ideal nested segmentation, every nested page table walk is replaced with a simple add operation. The nested seg (50 percent) configuration represents a less ideal case. With the configuration, only half of the guest physical memory is allocated in a contiguous memory region, which can be directly translated from the segment base. The rest of the guest physical memory is translated by a partial flat page table. We use the balloon driver supported by the Xen hypervisor to select the contiguous half of the guest physical memory. We set the limit of the balloon driver to 50 percent, and the guest OS will release the 50 percent of its memory to the balloon driver, after booting. The released region is mapped to the partial flat page table,

TABLE 4
Application Input Data and Parameters

SPECint Applications	
Application	gcc, mcf, sjeng, libquantum, omnetpp, astar, xalanbmk
Dataset	Reference input
Commercial Applications	
SPECjbb 2005	4 warehouses
RUBiS	Default bidding workload
(like ebay)	Apache, PHP, MySQL
OLTP	MySQL
OrderEntry	Swingbench (Oracle)
StressTest	Swingbench (Oracle)
KernelCompile	linux kernel 2.6.38
Volano	50 rooms, 1000 connections
(chatting server)	Apache, Sun-JVM

and the rest of memory is mapped to the segmented region. When an application starts to run, the released memory will be allocated again, but it will be mapped to the partial flat page table.

The pure nested segmentation reduces the execution times by 7 percent on average from the flat page table. Gcc, mcf, RUBiS, and Volano exhibit significant performance improvements with nested segmentation. The 50 percent segmentation shows similar performance improvements to the pure one, as their differences are marginal for most of the applications. Although nested segmentation may not be applicable for all the cases of virtualization, it can potentially provide performance improvements without any extra optimization to reduce the overheads of nested page walks.

7.3 Flat Nested Page Tables

In this section, we first evaluate how much performance improvement a flat nested page table can achieve, compared to the baseline state-of-the-art 2D page walker. Fig. 11 shows the normalized execution times with a flat page table. The flat nested page table, requiring a very minor addition to the current page walker logic, improves the performance for all the applications from the baseline. On average, the flat nested page table reduces the execution times by 5 percent (SPECint) and 8 percent (commercial) from 4-level 2D PWC+NTLB, since flat can reduce page walk steps by upto 15 steps from the baseline. The improvements are high in RUBiS and Volano, by 10 and 12 percent respectively due to their high TLB miss rates. The performance improvements by the flat page table may be relatively

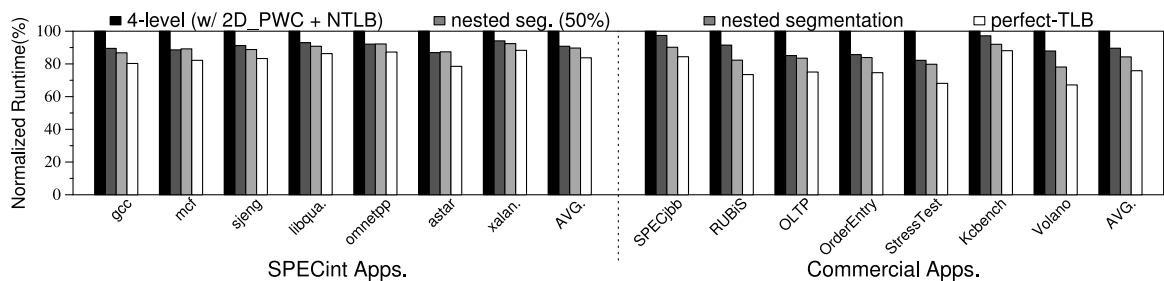


Fig. 10. Execution times of nested segmentation versus four-level nested page table (normalized to the baseline).

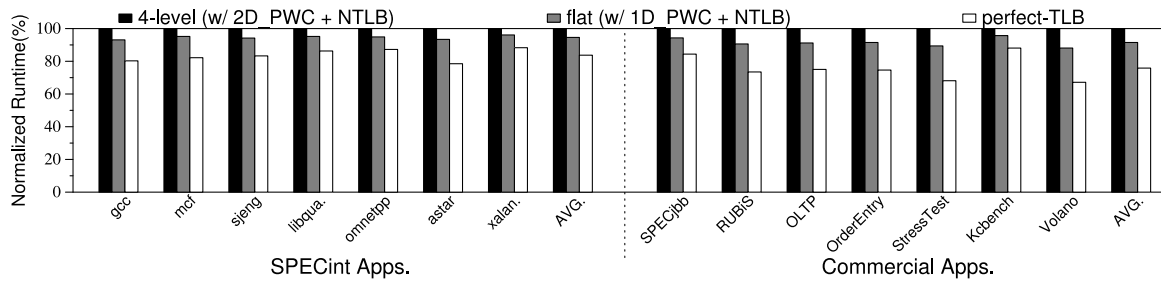


Fig. 11. Execution times of flat versus four-level nested page table (normalized to the baseline).

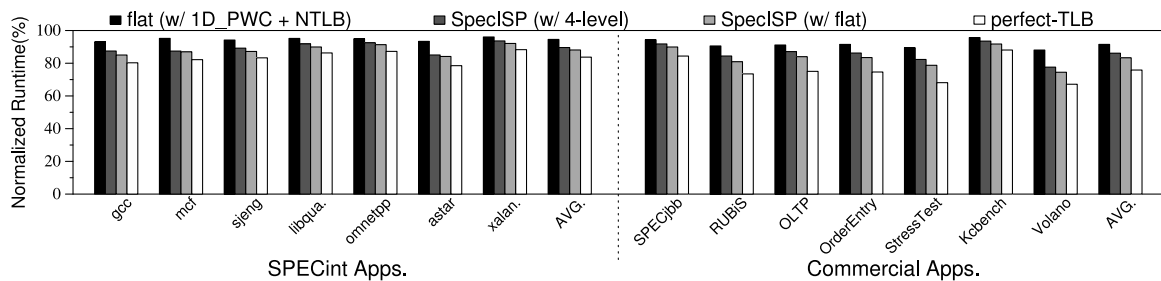


Fig. 12. Execution times of SpecISP versus flat nested paging (normalized to the baseline).

modest for SPECint, but the improvements are significant for the commercial workloads. The ideal perfect-TLB has further potential performance improvements of 11 percent for SPECint, and 16 percent for commercial from the flat nested page table.

7.4 Speculative Inverted Shadow Paging

In this section, we evaluate the performance benefit of speculative inverted shadow page tables. The inverted shadow page table reduces the number of references for guest page table lookups as well as nested page table lookups. Fig. 12 presents the normalized execution times with SpecISP. The inverted shadow paging with flat nested paging reduces the average execution times by 6 and 8 percent from flat, for SPECint and commercial respectively. Compared to baseline, it reduces the execution times by 12 and 17 percent. Also, the execution times with SpecISP become close to those with the perfect TLBs, with 4 and 7 percent performance differences for SPECint and commercial, from the ideal configuration.

Using traditional four-level nested page tables as the backing page tables for SpecISP reduces its effectiveness

slightly. However, we expect that by increasing the depth of speculative execution, SpecISP can potentially use either flat page tables or four-level page tables, achieving similar performance improvements.

Fig. 13 shows the cumulative distributions of latencies serving a TLB miss. Depending on whether references hit on the PWC or L2 cache, the TLB miss latencies vary. The figure shows two curves. The dotted line is the distribution of latencies for walking through the speculative shadow page table, and the solid line is for the non-speculative flat page table walk. The latencies through the inverted shadow paging are less than 12 cycles for more than 95 percent of TLB misses, as it requires a single lookup of the inverted shadow page table entry. The lookups commonly hit on the L2 cache. However, the latencies of non-speculative flat page table walks are within 60 cycles for 70 percent and 92 cycles for 90 percent of TLB misses. The latency differences are significant enough to justify accessing the inverted shadow paging speculatively, as speculative accesses return translations much faster than non-speculative accesses. Furthermore, the differences between two accesses are within 90 cycles for the majority of TLB misses. The differences are reasonably small enough to wait the verification by non-speculative accesses with the out-of-order execution capability in the current microprocessors.

Table 6 presents how often misspeculation occurs with the inverted shadow paging. For gcc, libquantum, and kernelCompile, there are relatively frequent changes of guest page tables, and thus misspeculation rates are high (2.1-5.3 percent). However, the rest of applications show very low rates of misspeculation. These low misspeculation rates indicate that the proposed speculative mechanism does not have significant overheads for pipeline flushes due to misspeculation. However, even if a misspeculation occurs, the cost of handling the misspeculation is much lower than the cost of hypervisor interventions in shadow paging.

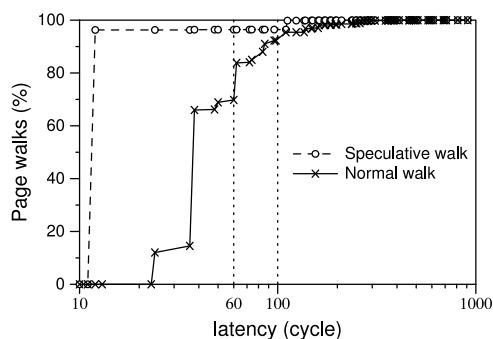


Fig. 13. Cumulative distributions of TLB miss latencies for the Volano benchmark.

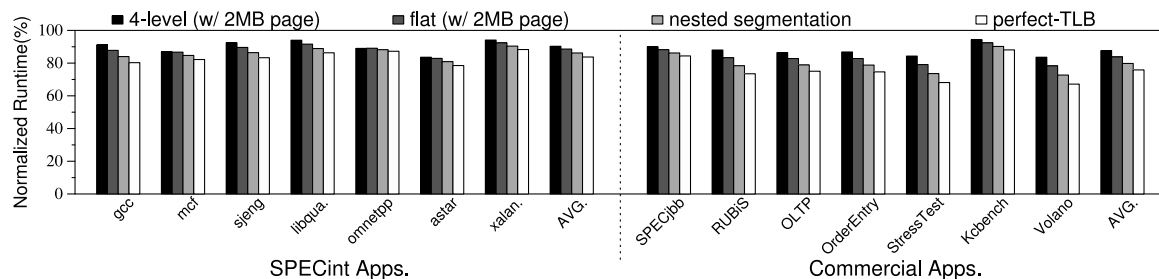


Fig. 14. Execution times for 2 MB large page: four-level NPT, flat NPT, and nested segmentation.

TABLE 5
Summary of Three Schemes

Type	Pros	Cons
Nested segmentation	Eliminate all nested page walks A small amount of additional hardware	Limit dynamic memory allocation Cannot support page sharing within a segment
Flat nested paging	Reduce many nested page walks Support fine-grained nested page tables No additional hardware	Require contiguous memory for nested tables
Speculative inverted shadow paging	Significant performance benefits	Increase the hardware complexity significantly

7.5 The Effect of Large Page

Increasing page size will improve overall performance because it skips the access to the last nested page table entries (nL1) in each level. The number of worst-case memory references can be reduced from 24 to 15. Fig. 14 shows the execution times when enabling large pages (2 MB) on the both guest and hypervisor level. The four-level with large page indicates the quite improved performance in most applications. Although the effectiveness of flat page table is likely to decrease, it also benefits from the large page when enabling the large page on a guest. In such a case, the performance gap between four-level and flat still exists since our flat page table also reduces the number of memory references from 9 to 7. The nested segmentation outperforms all others on nested page tables, since it does not require accessing nested page tables.

8 CONCLUSIONS

This paper explored how the nested address translation mechanism for virtualization can be improved, if the distinct characteristics of memory management for virtual machines are considered for the architectural support. We proposed and evaluated three schemes to reduce the overheads of nested address translation in virtualized systems.

TABLE 6
Misspeculation Rates with SpecISP

Workloads	Mis-spec. rate	Workloads	Mis-spec. rate
gcc	2.072 percent	SPECjbb	0.057 percent
mcf	0.008 percent	RUBiS	0.051 percent
sjeng	0.150 percent	OLTP	0.000 percent
libquantum	2.400 percent	OrderEntry	0.008 percent
omnetpp	0.000 percent	StressTest	0.005 percent
astar	0.000 percent	KernelCompile	5.312 percent
xalancbmk	0.672 percent	Volano	0.000 percent

The proposed three techniques have trade-offs between their performance benefits and hardware complexity required. Table 5 summarizes the advantages and disadvantages of the proposed schemes. The first two techniques, nested segmentation and flat page table, focus on how to efficiently translate a guest physical address to a system physical address. The nested segmentation can entirely eliminate the nested address translation, and it does not require a complex hardware because it can reuse the traditional segmentation hardware with minor modifications. The page-based address translation, flat nested paging, to allow a hypervisor to dynamically manage the system memory. It can be used in cases where the system does not find any contiguously large chunks in nested segmentation. Although two schemes can deliver the performance benefits by simplifying the nested address translation, these do not reduce the overhead of walking multi-level guest page tables to translate a guest virtual address to a guest physical address. To overcome the overhead of traversing a guest page table, speculative inverted shadow paging can provide the fast address translation method by skipping the guest page walks as well as nested page walks. However, to achieve the high performance on address translation, it significantly increases the hardware complexity, compared to the nested segmentation and flat nested paging. The exploration of the design space for fast two-level address translation will lead to efficient memory virtualization in different scenarios for using virtualization.

ACKNOWLEDGMENTS

This work was supported in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIP) (No. NRF-2010-0025511 and No. NRF-2013R1A2A2A01015514), and by the IT R&D program of MSIP/KEIT [10041313, UX-oriented Mobile SW Platform].

REFERENCES

- [1] Microsoft, "Windows Server 2012 R2: Server Virtualization Technical Overview", Whitepaper, 2013.
- [2] *UltraSPARC III Cu User Manual*, Sun Microsystems, Santa Clara, CA, USA, 2003.
- [3] *AMD-V Nested Paging*, Advanced Micro Devices, Sunnyvale, CA, USA, 2008.
- [4] K. Adams and O. Agesen, "A comparison of software and hardware techniques for x86 virtualization," in *Proc. 12th Int. Conf. Archit. Support Programming Languages Oper. Syst.*, 2006, pp. 2–13.
- [5] O. Agesen, A. Garthwaite, J. Sheldon, and P. Subrahmanyam, "The evolution of an x86 virtual machine monitor," *SIGOPS Oper. Syst. Rev.*, vol. 44, no. 4, pp. 3–18, Dec. 2010.
- [6] J. Ahn, S. Jin, and J. Huh, "Revisiting hardware-assisted page walks for virtualized systems," in *Proc. 39th Annu. Int. Symp. Comput. Archit.*, 2012, pp. 476–487.
- [7] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, and A. Warfield, "Xen and the art of virtualization," in *Proc. 19th ACM Symp. Oper. Syst. Principles*, 2003, pp. 164–177.
- [8] T. W. Barr, A. L. Cox, and S. Rixner, "Translation caching: Skip, don't walk (the page table)," in *Proc. 37th Int. Symp. Comput. Archit.*, 2010, pp. 8–59.
- [9] T. W. Barr, A. L. Cox, and S. Rixner, "SpecTLB: A mechanism for speculative address translation," in *Proc. 38th Int. Symp. Comput. Archit.*, 2011, pp. 307–318.
- [10] A. Basu, J. Gandhi, J. Chang, M. D. Hill, and M. M. Swift, "Efficient virtual memory for big memory servers," in *Proc. 40th Annu. Int. Symp. Comput. Archit.*, 2013, pp. 237–248.
- [11] A. Basu, M. D. Hill, and M. M. Swift, "Reducing memory reference energy with opportunistic virtual caching," in *Proc. 39th Annu. Int. Symp. Comput. Archit.*, 2012, pp. 297–308.
- [12] R. Bhargava, B. Serebrin, F. Spadini, and S. Manne, "Accelerating two-dimensional page walks for virtualized systems," in *Proc. 13th Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2008, pp. 26–35.
- [13] A. Bhattacharjee, "Large-reach memory management unit caches," in *Proc. 46th Int. Symp. Microarchit.*, 2013, pp. 383–394.
- [14] A. Bhattacharjee, D. Lustig, and M. Martonosi, "Shared last-level TLBs for chip multiprocessors," in *Proc. 17th Int. Symp. High Perform. Comput. Archit.*, 2011, pp. 62–63.
- [15] A. Bhattacharjee and M. Martonosi, "Characterizing the TLB behavior of emerging parallel workloads on chip multiprocessors," in *Proc. 18th Int. Conf. Parallel Archit. Compilation Tech.*, 2009, pp. 29–40.
- [16] A. Bhattacharjee and M. Martonosi, "Inter-core cooperative TLB for chip multiprocessors," in *Proc. 15th Int. Conf. Archit. Support Program. Languages and Oper. Syst.*, 2010, pp. 359–370.
- [17] E. Bugnion, S. Devine, K. Govil, and M. Rosenblum, "Disco: Running commodity operating systems on scalable multiprocessors," *ACM Trans. Comput. Syst.*, vol. 15, no. 4, pp. 412–447, Nov. 1997.
- [18] M. Butler, L. Barnes, D. D. Sarma, B. Gelinas, "Bulldozer: An Approach to Multithreaded Compute Performance," *IEEE Micro*, vol. 31, no. 2, pp. 6–15, 2011.
- [19] X. Chang, H. Franke, Y. Ge, T. Liu, K. Wang, J. Xenidis, F. Chen, and Y. Zhang, "Improving virtualization in the presence of software managed translation lookaside buffers," in *Proc. 40th Annu. Int. Symp. Comput. Archit.*, 2013, pp. 120–129.
- [20] S. Chaudhry, R. Cypher, M. Ekman, M. Karlsson, A. Landin, S. Yip, H. Zeffer, and M. Tremblay, "Simultaneous speculative threading: A novel pipeline architecture implemented in Sun's rock processor," in *Proc. 36th Annu. Int. Symp. Comput. Archit.*, 2009, pp. 484–495.
- [21] J. B. Chen, A. Borg, and N. P. Jouppi, "A simulation based study of TLB performance," in *Proc. 19th Annu. Int. Symp. Comput. Archit.*, 1992, pp. 114–123.
- [22] R. C. Daley and J. B. Dennis, "Virtual memory, processes, and sharing in multics," *Commun. ACM*, vol. 11, no. 5, pp. 306–312, May 1968.
- [23] Z. Fang, L. Zhang, J. B. Carter, W. C. Hsieh, and S. A. McKee, "Reevaluating online superpage promotion with hardware support," in *Proc. 7th Int. Symp. High Perform. Comput. Archit.*, 2001, pp. 63–72.
- [24] N. Ganapathy and C. Schimmel, "General purpose operating system support for multiple page sizes," in *Proc. Annu. Conf. USENIX Annu. Tech. Conf.*, 1998, pp. 91–104.
- [25] J. Gandhi, A. Basu, M. D. Hill, and M. M. Swift, "Efficient memory virtualization," in *Proc. 47th Int. Symp. Microarchit.*, 2014, pp. 178–189.
- [26] R. P. Goldberg, "Survey of virtual machine research," *IEEE Comput.*, vol. 7, no. 9, pp. 34–45, Sep. 1974.
- [27] J. Huck and J. Hays, "Architectural support for translation table management in large address space machines," in *Proc. 20th Int. Symp. Comput. Archit.*, 1993, pp. 39–50.
- [28] B. Jacob and T. Mudge, "Virtual memory in contemporary microprocessors," *IEEE Micro*, vol. 18, no. 4, pp. 60–75, Jul./Aug. 1998.
- [29] B. L. Jacob and T. N. Mudge, "A look at several memory management units, TLB-refill mechanisms, and page table organizations," in *Proc. 8th Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 1998, pp. 295–306.
- [30] G. B. Kandiraju and A. Sivasubramaniam, "Going the distance for TLB prefetching: An application-driven study," in *Proc. 29th Annu. Int. Symp. Comput. Archit.*, 2002, pp. 195–206.
- [31] D. Lustig, A. Bhattacharjee, and M. Martonosi, "TLB improvements for chip multiprocessors: Inter-core cooperative prefetchers and shared last-level TLBs," *ACM Trans. Archit. Code Optim.*, vol. 10, no. 1, pp. 2:1–2:38, Apr. 2013.
- [32] P. Magnusson, M. Christensson, J. Eskilson, D. Forsgren, G. Hallberg, J. Hogberg, F. Larsson, A. Moestedt, and B. Werner, "Simics: A full system simulation platform," *Computer*, vol. 35, no. 2, pp. 50–58, Feb. 2002.
- [33] C. McCurdy, A. Cox, and J. Vetter, "Investigating the TLB behavior of high-end scientific applications on commodity microprocessors," in *Proc. Int. Symp. Perform. Anal. Syst. Softw.*, 2008, pp. 95–104.
- [34] J. Navarro, S. Iyer, P. Druschel, and A. Cox, "Practical, transparent operating system support for superpages," in *Proc. 5th Symp. Oper. Syst. Design Implementation*, 2002, pp. 89–104.
- [35] B. Pham, A. Bhattacharjee, Y. Eckert, and G. Loh, "Increasing TLB reach by exploiting clustering in page translations," in *Proc. 20th Int. Symp. High Perform. Comput. Archit.*, 2014, pp. 558–567.
- [36] B. Pham, V. Vaidyanathan, A. Jaleel, and A. Bhattacharjee, "Colt: Coalesced large-reach TLBs," in *Proc. 45th Int. Symp. Microarchit.*, 2012, pp. 258–269.
- [37] T. H. Romer, W. H. Ohlrich, A. R. Karlin, and B. N. Bershad, "Reducing TLB and memory overhead using online superpage promotion," in *Proc. 22nd Annu. Int. Symp. Comput. Archit.*, 1995, pp. 176–187.
- [38] S. Srikantaiah and M. Kandemir, "Synergistic TLBs for high performance address translation in chip multiprocessors," in *Proc. 43rd Int. Symp. Microarchit.*, 2010, pp. 313–324.
- [39] I. Subramanian, C. Mather, K. Peterson, and B. Raghunath, "Implementation of multiple pagesize support in HP-UX," in *Proc. Annu. Conf. USENIX Annu. Tech. Conf.*, 1998, pp. 105–118.
- [40] M. Swanson, L. Stoller, and J. Carter, "Increasing TLB reach using superpages backed by shadow memory," in *Proc. 25th Annu. Int. Symp. Comput. Archit.*, 1998, pp. 204–213.
- [41] M. Talluri and M. D. Hill, "Surpassing the TLB performance of superpages with less operating system support," in *Proc. 6th Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 1994, pp. 171–182.
- [42] M. Talluri, M. D. Hill, and Y. A. Khalidi, "A new page table for 64-bit address spaces," in *Proc. 15th ACM Symp. Oper. Syst. Principles*, 1995, pp. 184–200.
- [43] M. Talluri, S. Kong, M. D. Hill, and D. A. Patterson, "Tradeoffs in supporting two page sizes," in *Proc. 19th Annu. Int. Symp. Comput. Archit.*, 1992, pp. 415–424.
- [44] R. Uhlig, G. Neiger, D. Rodgers, A. L. Santoni, F. C. Martins, A. V. Anderson, S. M. Bennett, A. Kagi, F. H. Leung, and L. Smith, "Intel virtualization technology," *IEEE Comput.*, vol. 38, no. 5, pp. 48–56, 2005.
- [45] VMware, "Performance evaluation of Intel EPT hardware assist," Tech. Rep. EN-000194-00, 2009.
- [46] VMware, "Understanding memory resource management in VMware ESX server," Tech. Rep. VMW_ESX_Memory_09Q3_WP_P20_R3, 2009.
- [47] C. A. Waldspurger, "Memory resource management in VMware ESX server," in *Proc. 5th Symp. Oper. Syst. Design Implementation*, 2002, pp. 181–194.
- [48] X. Wang, J. Zang, Z. Wang, Y. Luo, and X. Li, "Selective hardware/software memory virtualization," in *Proc. 7th Int. Conf. Virtual Execution Environ.*, 2011, pp. 217–226.



Jeongseob Ahn received the BS degree in computer science and engineering from Dongguk University in 2009 and the MS degree in computer science from KAIST in 2011. He is currently working toward the PhD degree in computer science at KAIST. His research interests include computer architecture, operating systems, and virtualization.



Jaehyuk Huh received the BS degree in computer science from Seoul National University, and the MS and PhD degrees in computer science from the University of Texas at Austin. He is currently an associate professor of computer science at KAIST. His research interests include computer architecture, parallel computing, virtualization, and system security.



Seongwook Jin received the BS degree in computer science and engineering from Kumoh National University in 2008 and the MS degree in computer science from KAIST in 2010. He is currently working toward the PhD degree in computer science at KAIST. His research interests include computer architecture, security, and virtualization.

▷ **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.**